



Chapter 24: Advanced Indexing

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Bitmap Indices

- Bitmap indices are a special type of index designed for efficient querying on multiple keys
- Records in a relation are assumed to be numbered sequentially from, say, 0
 - Given a number n it must be easy to retrieve record n
 - Particularly easy if records are of fixed size
- Applicable on attributes that take on a relatively small number of distinct values/intervals
 - E.g., gender, country, state, ...
 - E.g., income-level (income broken up into a small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000-infinity)
- A bitmap is simply an array of bits



Bitmap Indices (Cont.)

- In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute
 - Bitmap has as many bits as records
 - In a bitmap for value v , the bit for a record is 1 if the record has the value v for the attribute, and is 0 otherwise
 - Should keep bitmaps for all values, even null value

record number	<i>ID</i>	<i>gender</i>	<i>income_level</i>	Bitmaps for <i>gender</i>		Bitmaps for <i>income_level</i>	
		m		m	10010		
		f		f	01101	L1	10100
0	76766	m	L1			L2	01000
1	22222	f	L2			L3	00001
2	12121	f	L1			L4	00010
3	15151	m	L4			L5	00000
4	58583	f	L3				



Bitmap Indices (Cont.)

- Bitmap indices are useful for queries on multiple attributes
 - not particularly useful for single attribute queries
- Queries are answered using bitmap operations
 - Intersection (and), Union (or), Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap
 - E.g., $100110 \text{ AND } 110011 = 100010$
 $100110 \text{ OR } 110011 = 110111$
 $\text{NOT } 100110 = 011001$
 - Males with income level L1: $10010 \text{ AND } 10100 = 10000$
 - Can then retrieve required tuples.
 - Counting number of matching tuples is even faster
- Bitmap indices generally very small compared with relation size



Spatial and Temporal Indices



Spatial Data

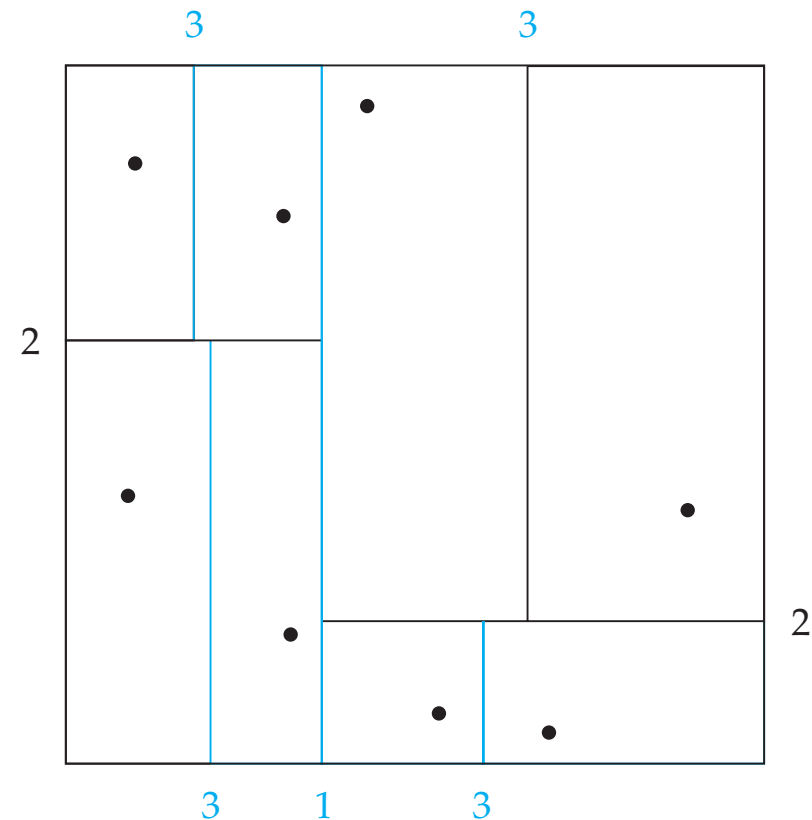
- Spatial data refers to data referring to a point or a region in two or higher dimensional space. For example,
 - The location of restaurants, identified by a (latitude, longitude) pair, is a form of spatial data.
 - The spatial extent of a farm or a lake can be identified by a polygon, with each corner identified by a (latitude, longitude) pair.
- Databases can store data types such as lines, polygons, in addition to raster images
 - Queries can use spatial conditions (e.g. contains or overlaps).
 - Queries can mix spatial and nonspatial conditions
- **Nearest neighbor queries**, given a point or an object, find the nearest object that satisfies given conditions.
- **Range queries** deal with spatial regions. e.g., ask for objects that lie partially or fully inside a specified region.
- Queries that compute **intersections or unions of regions**.
- **Spatial join** of two spatial relations with the location playing the role of join attribute.



Indexing of Spatial Data – K-D Tree

k-d tree - early structure used for indexing in multiple dimensions.

- Each level of a *k-d* tree partitions the space into two.
 - choose one dimension for partitioning at the root level of the tree.
 - choose another dimensions for partitioning in nodes at the next level and so on, cycling through the dimensions.
- In each node, approximately half of the points stored in the sub-tree fall on one side and half on the other.
- Partitioning stops when a node has less than a given number of points.

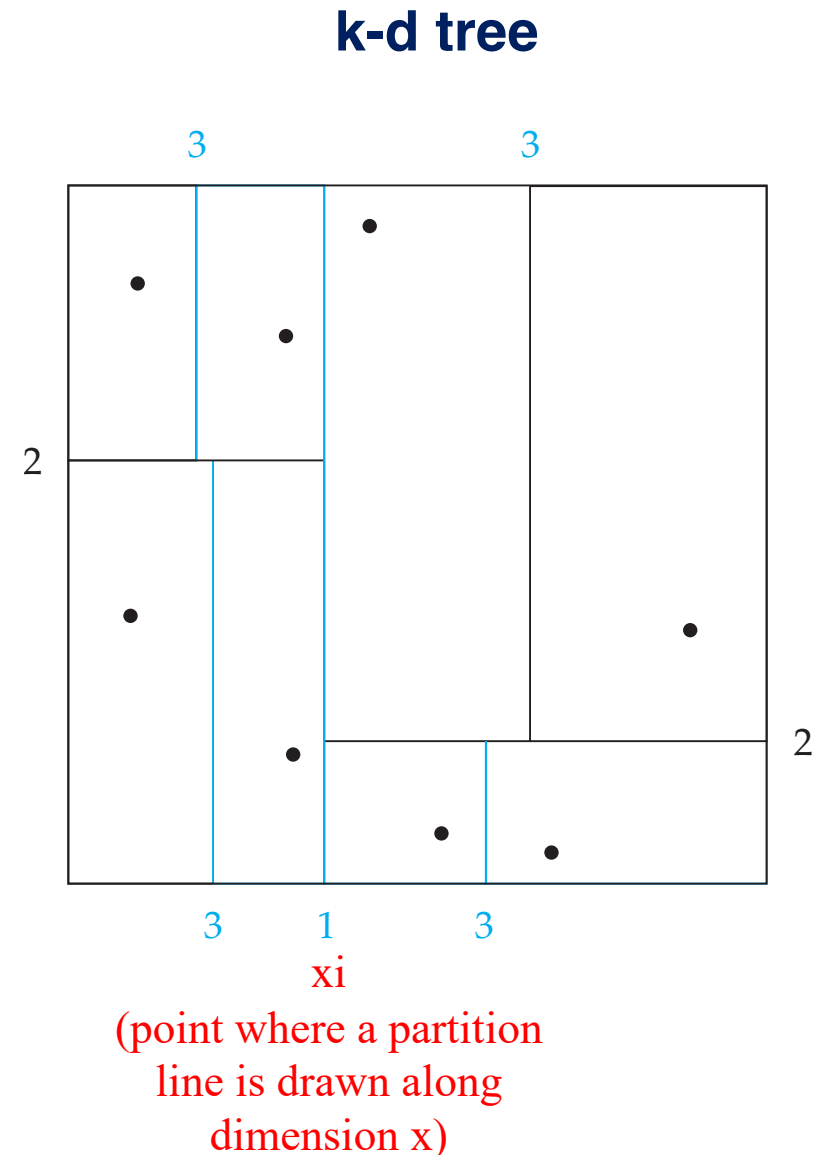


A k-d tree representation of a set of points on a 2-dimensionial space,



Indexing of Spatial Data – K-D Tree

- Rectangular range queries: ask for points within a specified rectangular region
 - specifies an interval on each dimension. E.g.,
 - x dimension lies between 50 and 80, and y dimension lies between 40 and 70.
- Range search can be performed recursively
 - If query range contains x_i , search is recursively performed on both children.
 - If query range is to the left of x_i , search is recursively performed only on the left child, and otherwise it is performed only on the right subtree.
 - If the node is a leaf, all entries that are contained in the query range are retrieved.

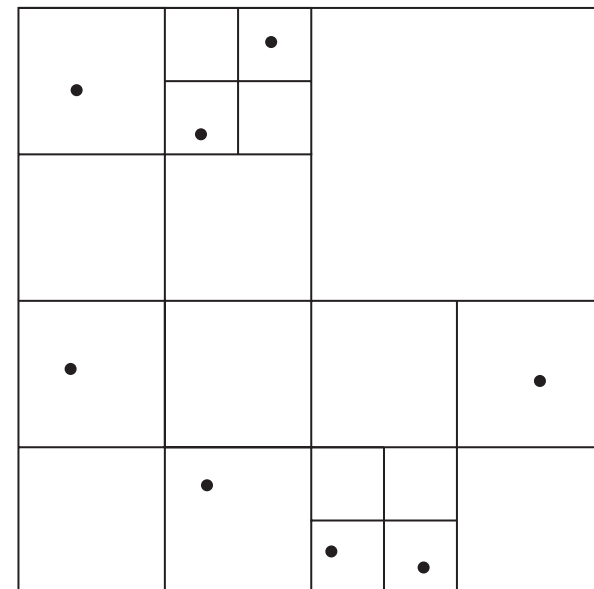




Division of Space by Quadrees

Quadrees

- Each node is associated with a rectangular region of space; the top node is associated with the entire target space.
- Each non-leaf nodes divides its region into four equal sized quadrants
 - correspondingly each such node has four child nodes corresponding to the four quadrants and so on
- Leaf nodes have between zero and some fixed maximum number of points (set to 1 in the example).





R-Trees

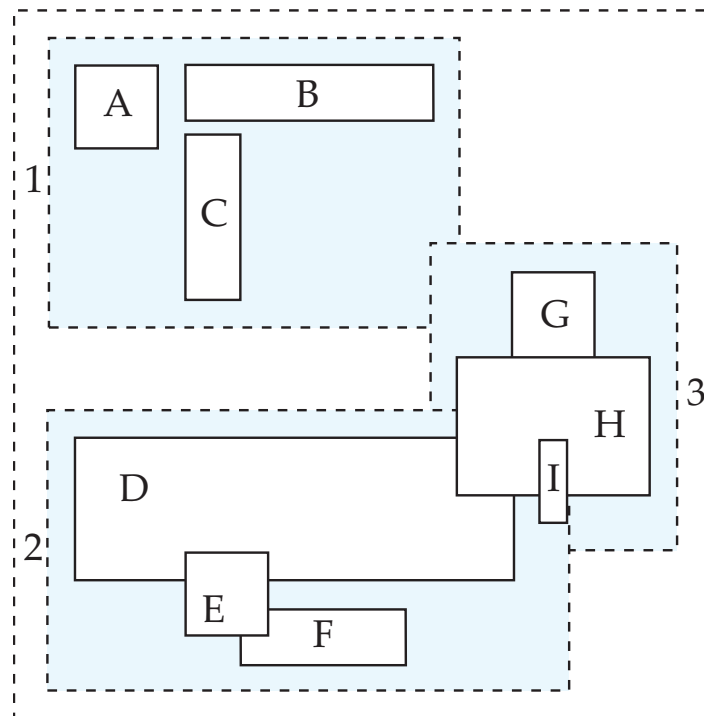
- **R-trees** are a N -dimensional extension of B^+ -trees, useful for indexing sets of rectangles and other polygons.
- Supported in many modern database systems, along with variants like R^+ -trees and R^* -trees.
- Basic idea: generalize the notion of a one-dimensional interval associated with each B^+ -tree node to an **N -dimensional interval**, that is, an N -dimensional rectangle.
- Will consider only the two-dimensional case ($N = 2$)
 - generalization for $N > 2$ is straightforward, although R-trees work well only for relatively small N
- A polygon is stored only in one node, and the bounding box of the node must contain the polygon



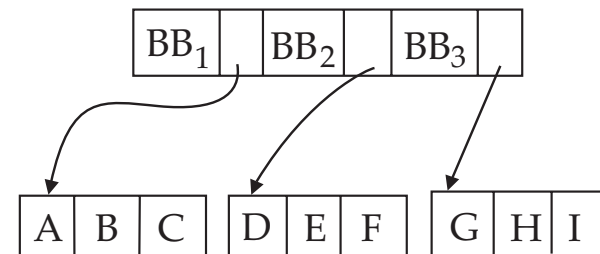
Example R-Tree

- The **bounding box** of a node is a minimum sized rectangle that contains all the rectangles/polygons associated with the node
 - *Bounding boxes for the children of a node are allowed to overlap*

Rectangles being Indexed



R-Tree

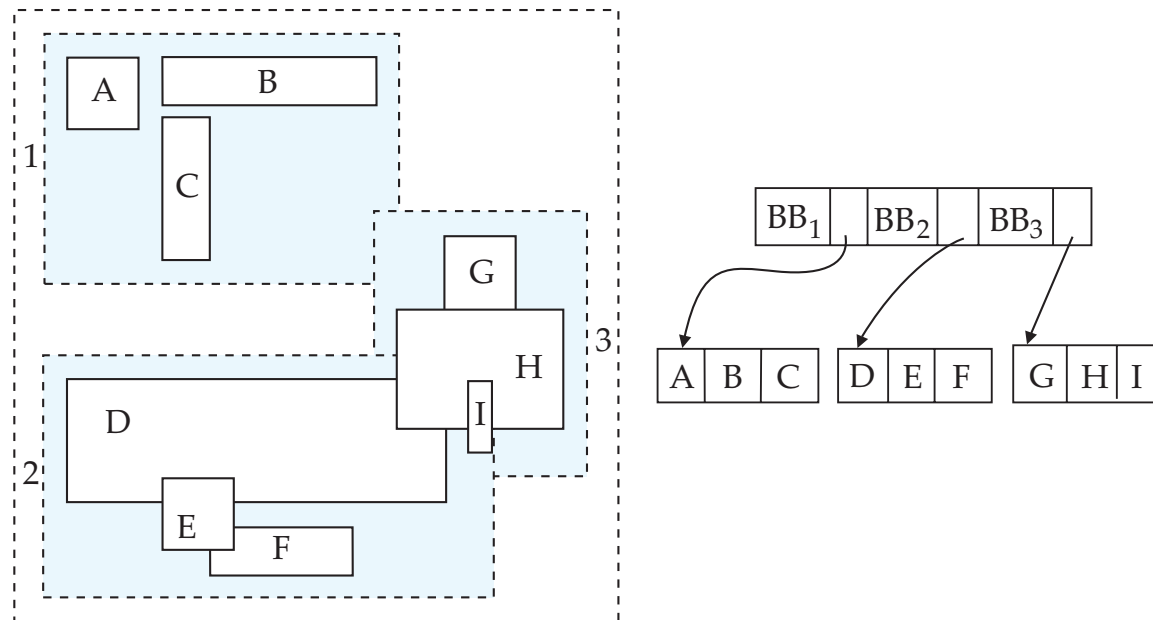


- rectangles (drawn with a solid line)
- bounding boxes (drawn with a dashed line)



Search in R-Trees

- To find data items intersecting a given query point/region, perform the following recursive procedure, starting from the root node:
 - If the node is a leaf node, output the data items whose keys intersect the given query point/region.
 - Else, for each child of the current node whose bounding box intersects the query point/region, recursively search the child





Indexing Temporal Data

- Temporal data refers to data that has an associated time period (interval)
- Time interval has a start and end time
 - End time set to infinity (or large date such as 9999-12-31) if a tuple is currently valid and its validity end time is not currently known
- Query may ask for all tuples that are valid at a point in time or during a time interval
 - Index on valid time period speeds up this task

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>start</i>	<i>end</i>
BIO-101	Intro. to Biology	Biology	4	1985-01-01	9999-12-31
CS-201	Intro. to C	Comp. Sci.	4	1985-01-01	1999-01-01
CS-201	Intro. to Java	Comp. Sci.	4	1999-01-01	2010-01-01
CS-201	Intro. to Python	Comp. Sci.	4	2010-01-01	9999-12-31

Figure 7.17 A temporal version of the *course* relation



Indexing Temporal Data (Cont.)

- To create a temporal index on attribute a :
 - Use spatial index, such as Quadtrees, with attribute a as one dimension, and time as another dimension
 - Valid time forms an interval in the time dimension
 - Tuples that are currently valid cause problems, since value is infinite or very large
 - Solution: store all current tuples (with end time as infinity) in a separate index, indexed on $(a, start-time)$
 - To find tuples valid at a point in time t in the current tuple index, search for tuples in the range $(a, 0)$ to (a, t)
- Temporal index on primary key can help enforce temporal primary key constraint

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>start</i>	<i>end</i>
BIO-101	Intro. to Biology	Biology	4	1985-01-01	9999-12-31
CS-201	Intro. to C	Comp. Sci.	4	1985-01-01	1999-01-01
CS-201	Intro. to Java	Comp. Sci.	4	1999-01-01	2010-01-01
CS-201	Intro. to Python	Comp. Sci.	4	2010-01-01	9999-12-31



End of Chapter 24